

Explainable Chronos: A Post-Hoc Framework for Faithful Verbalization and Interactive Querying of Probabilistic Time-Series Forecasts

[Author 1], [Author 2], [Author 3], [Author 4], Emanuele Romito

Course: 01VIXSM — Deep Natural Language Processing, A.Y. 2025–2026

Repository: <https://github.com/Voyagers-time-series-forecasting/explainable-chronos>

Abstract—

Index Terms—

I. INTRODUCTION

Probabilistic time-series forecasting models such as Chronos-2 [1] produce quantile predictions (P10, P50, P90) that encode rich information about expected trends, uncertainty, and tail risks. However, these numerical outputs are difficult for non-expert stakeholders to interpret: a vector of quantile values communicates nothing intuitively about *why* a forecast was generated, how confident the model is, or what role each input covariate played in shaping the prediction. This gap between model output and human understanding is precisely what our work is trying to fill.

Our project addresses this gap by building a modular NLP framework that sits entirely on top of Chronos-2 at inference time and produces two complementary capabilities: (i) automatic natural-language explanations of forecasts grounded in extracted numerical features and covariate attributions (Extension 1), and (ii) an interactive dialogue interface through which users can query and counterfactually modify those forecasts using plain English (Extension 2).

Extension 1 addresses two complementary challenges: generating natural-language summaries that faithfully describe the content of quantile forecasts, and automatically verifying whether the generated text is consistent with the underlying numerical evidence. We frame the verification task as a Natural Language Inference (NLI) problem [2], [3], where a structured rendering of the numerical features serves as the *premise* and each verbalized sentence serves as the *hypothesis*. Our discourse planning stage draws on Rhetorical Structure Theory (RST) [7], using rhetorically motivated relations to organize the generated content into coherent multi-sentence summaries.

II. PROBLEM STATEMENT

A. Extension 1

Input. A historical time series $\mathbf{x} = (x_1, \dots, x_T) \in \mathbb{R}^T$ and, optionally, a matrix of past covariates $\mathbf{Z} \in \mathbb{R}^{T \times C}$ with C named covariates.

Task. Given a Chronos-2 quantile forecast $\hat{\mathbf{y}}^{(q)}$ for quantiles $q \in \{0.10, 0.50, 0.90\}$ over a horizon H :

$$\hat{\mathbf{y}}^{(q)} = (\hat{y}_1^{(q)}, \dots, \hat{y}_H^{(q)}), \quad q \in \{0.10, 0.50, 0.90\}, \quad (1)$$

produce a natural-language summary $\mathcal{S} = (s_1, \dots, s_K)$ such that each sentence s_k is factually grounded in a subset of numerical features derived from the quantiles, and then score each sentence for factual consistency via NLI.

Output. A verbalization result comprising: (a) the paragraph-level summary $\mathcal{S}$; (b) per-sentence grounding meta-data mapping each s_k to the numerical features that generated it; (c) a consistency report containing per-sentence NLI entailment scores and an aggregate mean score indicating whether the summary is factually consistent.

III. METHODOLOGY

A. Extension 1

The pipeline consists of four sequential stages: forecast and feature extraction (Stage A), optional covariate attribution (Stage B), verbalization (Stage C), and NLI consistency scoring (Stage D).

B. Stage A: Forecast and Feature Extraction

The Chronos-2 model produces probabilistic forecasts from which we extract quantiles P10, P50, and P90. All subsequent feature extraction uses only NumPy and SciPy.

Trend direction and magnitude. A linear fit $\hat{y}_t^{(0.5)} = \alpha + \beta t$ yields the slope β . The normalized slope is computed as $\bar{\beta} = \beta / |\overline{P50}|$. Magnitude is classified using two thresholds: $|\bar{\beta}| > 0.05 \Rightarrow \text{sharply}$, $|\bar{\beta}| > 0.02 \Rightarrow \text{moderately}$, otherwise *slightly*. Direction is *flat* when $|\bar{\beta}| \leq 0.02$, *rising* when $\bar{\beta} > 0.02$, and *falling* otherwise.

Uncertainty profile. The prediction interval width $w_t = \hat{y}_t^{(0.9)} - \hat{y}_t^{(0.1)}$ yields relative uncertainty $\bar{w} = \bar{w} / |\overline{P50}|$, classified as *high* ($\bar{w} > 0.30$), *moderate* ($\bar{w} > 0.10$), or *low*. A linear fit to w_t determines whether intervals are *widening* (slope > 0.01), *narrowing* (slope < -0.01), or *stable*.

Interval asymmetry. We compute:

$$A = \frac{(P90 - P50) - (P50 - P10)}{\bar{w}}, \quad (2)$$

labelling as *symmetric* ($|A| < 0.10$), *right-skewed* ($A > 0$), or *left-skewed*.

Tail risk flags. Downside risk is flagged when $\min(P10) < x_T \cdot 0.80$; upside potential when $\max(P90) > x_T \cdot 1.20$.

Regime-shift detection. A Welch t -test compares the first and second halves of the P50 forecast; a p -value below 0.05 flags a structural break.

C. Stage B: Covariate Attribution

When covariates are available, the pipeline can attach attribution metadata to the verbalized forecast. We support two post-hoc attribution methods: a surrogate-model SHAP explainer and an attention-rollout explainer. Both methods return a ranked list of covariates with relative impact scores, and the top- k attributions (default $k = 5$) are passed to the verbalizer to generate attribution-aware sentences.

Surrogate model with SHAP. The first method trains a surrogate XGBoost regressor [12] to approximate the local mapping from covariate history to the P50 forecast level. Rolling windows over the covariate history produce multiple training rows with per-covariate aggregate features: mean, standard deviation, last value, minimum, and maximum. After fitting the surrogate, SHAP TreeExplainer is used to estimate feature contributions according to the Shapley-value framework of Lundberg and Lee [11]. The five aggregate-feature attributions belonging to the same covariate are summed by absolute value, yielding one importance score per covariate. This method is model-agnostic with respect to Chronos-2: it explains the surrogate approximation of the forecast behavior rather than the internal transformer computation directly.

Attention Rollout. The second method uses attention weights extracted from Chronos-2 and aggregates them using Attention Rollout, following Abnar and Zuidema’s method for quantifying attention flow in Transformers [13]. For each transformer layer, attention matrices are averaged across heads, augmented with an identity connection to account for residual paths, row-normalized, and then multiplied across layers. The resulting rollout matrix approximates how information flows through stacked self-attention layers. We aggregate this rollout attention over the token positions associated with each covariate and normalize the resulting scores into relative covariate impacts. Unlike the SHAP surrogate, this method uses internal attention signals from the forecasting model, but its interpretation depends on how Chronos-2 represents covariates in the input sequence.

D. Stage C: Verbalization

We implement two complementary approaches.

Approach A — Template Verbalizer. An RST-based sentence planner uses phrase-variant dictionaries and a DiscoursePlanner module. The planner inspects the extracted features and optional attributions to select rhetorical relations. Five relation types are supported:

- **ELABORATION:** activated when a regime shift is detected, providing the p -value detail.
- **CAUSE:** activated when the top attributed covariate contributes $> 20\%$, expressing the driver.
- **CONTRAST:** activated when (a) a non-flat trend co-occurs with high uncertainty, or (b) the top two covariates have opposing directions with the second contributing $> 15\%$.
- **CONCESSION:** activated when a rising trend co-occurs with downside risk.
- **SEQUENCE:** reserved for multi-phase forecasts.

Each sentence is paired with a grounding dictionary that records the exact numerical features (type, values, labels) from which it was generated. Phrase variants are sampled from pre-defined dictionaries using a seeded random generator for reproducibility.

Approach B — LLM Verbalizer. The template output serves as a draft for a local LLM. The LLM receives: (i) the numerical features, (ii) structured grounding triples of the form (subject, predicate, object), (iii) the template draft summary, and (iv) strict NLI-oriented constraints forbidding causal language, domain-specific speculation, and mathematical reasoning. Two LLM variants are evaluated: *LLM Guided* receives both the template draft and grounding triples; *LLM Raw* receives only the numerical features and grounding triples, without any template draft. For both LLM variants, each generated sentence is mapped to a *combined* grounding that aggregates all feature groundings from the template result, since the LLM may freely reorganize information across sentences.

E. Stage D: NLI Consistency Scoring

For each sentence s_k in the verbalization, a *premise* π_k is rendered from its grounding dictionary into unambiguous English. The rendering is type-specific: trend groundings produce statements about slope values and classifications; uncertainty groundings produce statements about interval widths and relative uncertainty; attribution groundings produce statements about SHAP values and directional effects.

The NLI model `facebook/bart-large-mnli` [4] is used via the HuggingFace zero-shot-classification pipeline to estimate:

$$P(\text{entail} \mid \pi_k, s_k). \quad (3)$$

The overall consistency score for summary S is the arithmetic mean:

$$\text{Score}(S) = \frac{1}{K} \sum_{k=1}^K P(\text{entail} \mid \pi_k, s_k). \quad (4)$$

A summary is deemed *consistent* when $\text{Score}(S) \geq 0.70$.

IV. EVALUATION PROTOCOL

Following the recommendation to distinguish between two types of ground truth, our evaluation protocol cleanly separates two orthogonal questions: (1) *Is the forecast itself accurate?* and (2) *Is the explanation faithful to what the model actually predicted?* A high-scoring explanation of an inaccurate forecast is still a correct explanation; conversely, a fluent but

hallucinated summary of an accurate forecast is a faithfulness failure.

A. Ground Truth 1: Forecast Accuracy

This axis evaluates the Chronos-2 model against observed actuals and is independent of the verbalization pipeline.

MASE (Mean Absolute Scaled Error). Defined as the ratio of the forecast’s mean absolute error to that of a naïve seasonal baseline:

$$\text{MASE} = \frac{\frac{1}{H} \sum_{t=1}^H |y_t - \hat{y}_t^{(0.5)}|}{\frac{1}{T-1} \sum_{t=2}^T |x_t - x_{t-1}|}. \quad (5)$$

A MASE below 1 indicates that the model outperforms the naïve baseline.

Prediction Interval Coverage (%). The percentage of actual future values that fall within the P10–P90 prediction interval:

$$\text{Coverage} = \frac{100}{H} \sum_{t=1}^H \mathbf{1}[\hat{y}_t^{(0.1)} \leq y_t \leq \hat{y}_t^{(0.9)}]. \quad (6)$$

For well-calibrated 80% prediction intervals, the expected coverage is 80%.

Interval Sharpness. The mean interval width normalized by the range of the actual values:

$$\text{Sharpness} = \frac{\hat{y}^{(0.9)} - \hat{y}^{(0.1)}}{\max(\mathbf{y}) - \min(\mathbf{y})}. \quad (7)$$

Lower sharpness indicates tighter, more informative intervals (desirable when coverage is maintained).

B. Ground Truth 2: Explanation Faithfulness

This axis evaluates whether the generated text accurately describes what the model predicted, regardless of whether the prediction itself was correct.

Per-Sentence NLI Entailment Score. For each sentence s_k , the entailment probability $P(\text{entail} \mid \pi_k, s_k) \in [0, 1]$ measures how strongly the NLI model judges the sentence to be supported by its numerical premise. Scores near 1 indicate strong factual grounding; scores below 0.5 suggest the sentence introduces claims not entailed by the data.

Overall Consistency Score. The arithmetic mean of per-sentence entailment probabilities across all K sentences. A threshold of 0.70 defines the binary pass/fail boundary.

Quantile Round-Trip Check. An exact-match verification that re-derives all feature labels (trend direction, magnitude, uncertainty level) directly from the raw P10/P50/P90 arrays and compares them against the labels used during verbalization. Any mismatch is recorded as a *faithfulness error*, indicating a bug in the feature extraction logic. This metric is binary per-scenario: zero mismatches (pass) or at least one (fail).

C. Illustrative Success and Failure Cases

To provide concrete intuition for the metrics, we describe representative cases from preliminary experiments on synthetic data.

Success case (Template verbalizer, trending scenario). The template generates: “*The forecast indicates that values are expected to remain relatively stable over the next 7 periods.*” The NLI premise reads: “*The median forecast slope is +0.0825 per period. The normalised slope is +0.0010. The trend is classified as slightly flat.*” The NLI model assigns an entailment probability of 0.999, because the sentence’s “relatively stable” directly mirrors the “slightly flat” classification.

Failure case (LLM Raw verbalizer, flat scenario). The LLM generates: “*The financial performance has been consistent but not exceptionally strong or weak, indicating potential risks associated with current market conditions.*” The NLI premise contains only slope values and classification labels—no mention of “financial performance” or “market conditions.” The NLI model assigns an entailment probability of 0.14, correctly flagging the hallucinated domain-specific content.

Failure case (Template verbalizer, regime-shift premise). The template generates: “*Additionally, a structural break is detected near the midpoint, indicating a potential change in the underlying pattern ($p=0.0131$).*” The NLI premise reads: “*A Welch t-test comparing the first and second halves yields a p-value of 0.0131, indicating a statistically significant mean shift.*” The entailment probability drops to 0.011 because the NLI model struggles to link “structural break” with “statistically significant mean shift”—a known lexical sensitivity of NLI models.

V. EXPERIMENTS

A. Data Description

We evaluate on four real-world time-series datasets: ETTh1, ETTm1, Seattle Weather, and S&P 500. These datasets provide observed actuals against which both forecast accuracy and explanation faithfulness can be assessed, and they include covariates for evaluating attribution-aware verbalization.

B. Experimental Design

Hardware. CPU-only execution (experiments in progress).

Software and libraries. Python 3.10+; chronos-forecasting (Chronos-2-small, autogluon/chronos-2-small); transformers (BART-large-MNLI for NLI, Qwen1.5-1.8B-Chat for LLM verbalization); xgboost (surrogate model); shap (TreeExplainer); NumPy, SciPy, pandas, matplotlib.

Validation method. The dual evaluation protocol from Section V is applied: forecast accuracy metrics (MASE, coverage, sharpness) are computed against observed actuals; explanation faithfulness metrics (NLI consistency, round-trip check) are computed against the model’s own numerical features.

Planned comparisons. Three verbalizer types (Template, LLM Guided, LLM Raw) are compared across the selected real-world datasets, with and without covariate attribution.

C. Results

Results will be reported after completion of the M5 experiments.

VI. CONCLUSIONS AND TAKEAWAYS

To be completed after experimental results are available.

REFERENCES

- [1] A. F. Ansari, L. Stella, C. Turkmen, et al., “Chronos-2: From Univariate to Universal Forecasting,” *arXiv preprint arXiv:2510.15821v1*, 2025.
- [2] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, “A large annotated corpus for learning natural language inference,” in *Proc. EMNLP*, 2015, pp. 632–642.
- [3] A. Williams, N. Nangia, and S. R. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in *Proc. NAACL-HLT*, 2018, pp. 1112–1122.
- [4] M. Lewis, Y. Liu, N. Goyal, et al., “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” in *Proc. ACL*, 2020, pp. 7871–7880.
- [5] O. Dušek and Z. Kasner, “Evaluating Semantic Accuracy of Data-to-Text Generation with Natural Language Inference,” in *Proc. INLG*, 2020, pp. 131–137.
- [6] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proc. EMNLP*, 2020, pp. 9332–9346.
- [7] W. C. Mann and S. A. Thompson, “Rhetorical Structure Theory: Toward a functional theory of text organization,” *Text*, vol. 8, no. 3, pp. 243–281, 1988.
- [8] M. Taboada and W. C. Mann, “Applications of Rhetorical Structure Theory,” *Discourse Studies*, vol. 8, no. 4, pp. 567–588, 2006.
- [9] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial Intelligence*, vol. 63, no. 1–2, pp. 341–385, 1993.
- [10] D. Rösner and M. Stede, “Customizing RST for the automatic production of technical manuals,” in *Aspects of Automated Natural Language Generation* (LNAI 587), Springer, 1992, pp. 199–214.
- [11] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [12] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. KDD*, 2016, pp. 785–794.
- [13] S. Abnar and W. Zuidema, “Quantifying Attention Flow in Transformers,” in *Proc. ACL*, 2020, pp. 4190–4197.
- [14] J. Bai, S. Bai, Y. Chu, et al., “Qwen Technical Report,” *arXiv preprint arXiv:2309.16609*, 2023.